An Analysis of Several Autoencoder Configurations to Identify Galaxy Mergers via Anomaly Detection

Sarthak Kumar
ksarthak962@gmail.com

August 2026

Abstract

The Euclid Quick Data Release recently released a catalog of galaxy images from the mission and developed a Convolutional Neural Network (CNN) to identify galaxy mergers [1, 2]. However, many of the galaxies were not distinctly identified. In this paper, I aim to classify these remaining "unknown" galaxies using Autoencoders designed for Anomaly Detection. I evaluate configurations of several different models, image denoising methods, and loss functions using the AUC-PR, AUC-ROC, and F1-score metrics. I found that using a Contractive Autoencoder, VisuShrink wavelet denoising, and a Gaussian-weighted χ^2-Loss Function provided the largest AUC-PR Score. This specific configuration was used to classify 2763 of the unknown galaxies.

1 Introduction

Galaxy Mergers are important astronomical events in which two (or more) galaxies gravitate toward each other, joining into one. They are important phenomena to study because they are linked to triggering active galactic nuclei (AGNs) and have long been known to drive galactic evolution [1]. As such, it is of principal importance that there exist data catalogs that are readily available for research usage. The Euclid Collaboration Team has already released their catalog of galaxy mergers. The issue is that around 36.2% of the ∼560,000 galaxies in the catalog are marked as unclassified. In order to reduce the number of unknown galaxies, I tested different configurations of machine learning models. The best one can then reliably be used in the future to classify the unknown galaxies.

To classify the galaxies, I used a process known as Anomaly Detection. Anomaly Detection works by identifying the outliers in a dataset and marking them as anomalous. For the purpose of this paper, an anomaly would be any image that depicts a galaxy merger, whereas a normal image would simply depict a non-merging galaxy. There are multiple ways to detect anomalies, one of which is by utilizing a type of neural network known as a Generative Adversarial Network (GAN). GANs work by having two models work against each other: a generator and a discriminator. The generator synthesizes fake data, while the discriminator learns to identify the fake data. By training a GAN on solely normal data, the reconstruction errors for anomalous data are abnormally high, allowing them to be easily identified [3]. It has already been shown that GANs can be used in Anomaly Detection for Astronomical images, but the network itself is prone to model collapse [4].

Therefore, I opted to use Autoencoders to detect anomalies instead. An autoencoder has two functions: an encoder and a decoder. The input data is first passed through the encoder and is stored as a compressed representation, known as the latent representation. Then, the representation is passed through the decoder, where the original image is reconstructed (See Figure 1) [5]. For Anomaly Detection, the Autoencoders are trained on a dataset containing just the normal data. As seen in Figure 2, when anomalous data is passed to the trained autoencoder, the reconstruction errors are higher because the model has not encountered anomalies yet [6]. This allows for images with a reconstruction error higher than a certain threshold to be selected as anomalies.

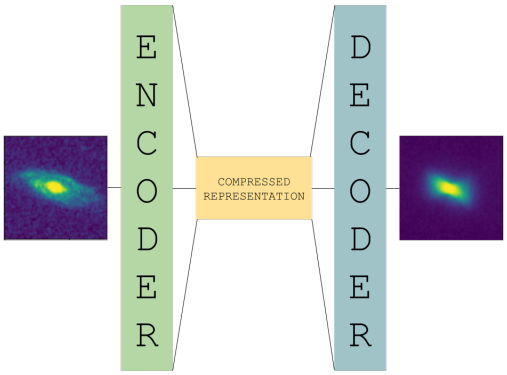

Figure 1: Diagram of the Autoencoder architecture.

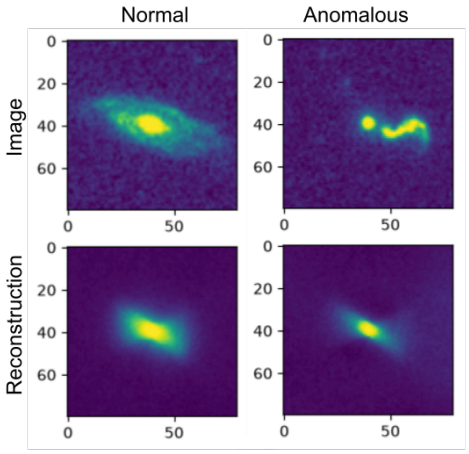

Figure 2: A table showing the reconstructions for a normal galaxy and an anomalous one.

2 Data

2.1 Data Sourcing

Images were sourced from the Euclid Quick Data Release (Q1) using the `AstroQuery` Euclid archive and the Galaxy Merger Catalog provided by La Marca et al. (2025) [7, 8]. For the normal dataset, galaxies were selected such that they were both flagged as non-mergers by the catalog and had a predicted merger probability $< 10\%$. These criteria minimized contamination in the normal dataset from falsely classified merging galaxies. Each galaxy's right ascension and declination were used to create a `SkyCoord` object using `AstroPy` [9–11]. To best replicate the images used by La Marc et al. (2025), I made requests for 80×80 pixel cutouts from the Visible Imager on the telescope [1, 12]. These images were preprocessed by selecting for the middle 98th percentile interval and by applying a linear stretch to them. For the anomalous images, the above process was repeated but for galaxies that were marked as anomalous and had a merger probability $> 85.5\%$. The unknown galaxies were also selected. There were 2875 images of normal galaxies, 2824 images of anomalous galaxies, and 2763 images of unknown galaxies. The images were then separated into a training dataset and a testing dataset, with 80% of the images going in the training set and 20% going in the testing set. Only the normal images were used to train each configuration. Each configuration used the exact same training and testing sets. Example images can be seen in Figures 3, 4, and 5.

Figure 3: Example of a normal galaxy.

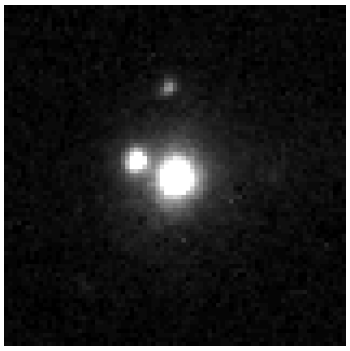

Figure 4: Example of an anomalous galaxy.

Figure 5: Example of an unknown galaxy.

2.2 Denoising Methods

As seen in Figures 3, 4, and 5, the images can be extremely noisy, and this noisiness is not the same for all images. Noise can artificially inflate the reconstruction error, leading to the model making

false predictions. To remedy this, the images will be denoised using a variety of methods. However, a balance needs to be struck, as denoising too much can hide key morphological features that would have helped the model identify mergers [13]. Therefore, these methods will be compared in order to use the most balanced denoising method on the unknown galaxies. The methods are compared in Figure 6.

Gaussian Denoising Gaussian filtering is a method of denoising that uses a Gaussian curve in two dimensions to filter data. A Gaussian curve in two dimensions is simply the product of two Gaussian functions; therefore, its equation is found by

$$G_\sigma(x, y) = g_\sigma(x) \times g_\sigma(y) \tag{1}$$

$$g_\sigma(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{x^2}{2\sigma^2}} \tag{2}$$

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \tag{3}$$

where x is the horizontal distance from the origin, y is the vertical distance from the origin, and σ is the standard deviation of the distribution [14]. A higher standard deviation corresponds to a steeper curve and more aggressive denoising, while a lower standard deviation corresponds to a flatter curve and less aggressive denoising. I denoised the images at $\sigma = 0.5, 1.0, 2.0$. using `SciPy`'s `ndimage` package [15].

Wavelet Denoising Wavelet denoising involves transforming the image data into a set of short, localized wavelets. A Discrete Wavelet Transform (DWT) is applied to the image in order to convert it into a set of wavelets. Similar to the frequency domain of a Fourier Transform, the wavelet domain contains values that, when large, indicate actual data but, when small, indicate noise. A high threshold removes all of the noise, but risks data loss as well. On the other hand, a low threshold removes some of the noise but keeps the data intact. There are two ways to choose thresholds: VisuShrink and BayesShrink. VisuShrink denoising chooses a universal threshold and sets any value below that threshold to 0. A reverse transform is then applied to construct the denoised image [14]. On the other hand, BayesShrink denoising computes different thresholds for each wavelet subband [16]. I denoised the images using both thresholding methods with `scikit-image`'s `denoise_wavelet()` function [17].

Noise2Void I implemented a Deep Learning approach to image denoising known as Noise2Void (N2V). Unlike other deep learning denoising algorithms, N2V does not require clean data or pairs of noisy images. Instead, it learns to remove noise by training CNNs on noisy images by hiding specific pixels within the image. The model learns to reconstruct those pixels based on the pixels surrounding them. In this way, when the model encounters noisy pixels within an image, it can reconstruct those pixels without noise by comparing them to the pixels surrounding them [18]. I used Noise2Void on my images with the implementation developed by Krull et al. (2018).

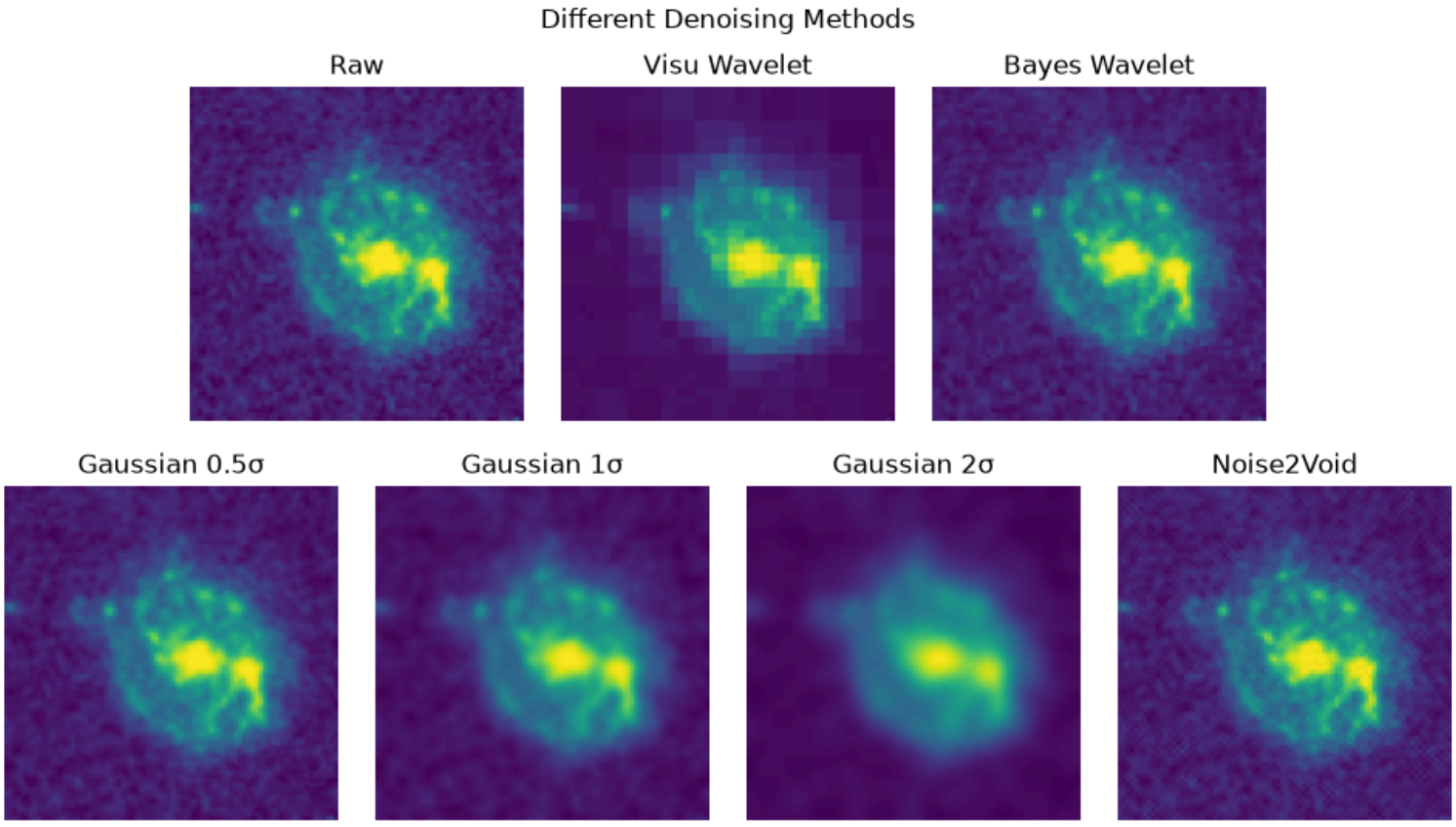

Figure 6: Different denoising methods applied to the same image of a galaxy. Note: While the image is grayscale for usage with the Autoencoder, the representation here is in color to discern morphological features.

Vanilla Autoencoder	Layer	Input Shape	Neurons	Activation Function	Output Shape
Encoder	Layer 1	(80, 80, 1)			6400
	Layer 2	6400	1024	ReLU	1024
	Layer 3	1024	512	ReLU	512
Latent Representation	Layer 4	512	128	ReLU	128
Decoder	Layer 1	128	512	ReLU	512
	Layer 2	512	1024	ReLU	1024
	Layer 3	1024	6400	Sigmoid	6400
Reconstruction	Layer 4	6400			(80, 80, 1)

Table 1: Vanilla Autoencoder Architecture

Conv.	Layer	Input	No. Kernels	Kernel Size	Stride	Activation Function	Output
Encoder	Layer 1	(80, 80, 1)	32	3	2	ReLU	(40, 40, 32)
	Layer 2	(40, 40, 32)	64	3	2	ReLU	(20, 20, 64)
	Layer 3	(20, 20, 64)	128	3	2	ReLU	(10, 10, 128)
	Layer 4	(10, 10, 128)					12800
Latent Representation	Layer 5	12800					4
Decoder	Layer 1	4					12800
	Layer 2	12800					(10, 10, 128)
	Layer 3	(10, 10, 128)	128	3	2	ReLU	(20, 20, 128)
	Layer 4	(20, 20, 128)	64	3	2	ReLU	(40, 40, 64)
	Layer 5	(40, 40, 64)	32	3	2	ReLU	(80, 80, 32)
Reconstruction	Layer 6	(80, 80, 32)	1	3	1	Sigmoid	(80, 80, 1)

Table 2: Convolutional Autoencoder Architecture

3 Methodology

3.1 Models

Vanilla Autoencoder The Vanilla Autoencoder is the most basic form of the Autoencoder. It consists of a basic encoder which condenses the input image into its latent representation and a decoder which reconstructs the original image from the latent representation (Figure 1). All layers used a rectified linear unit (ReLU) as an activation function, except for the last layer, which used a sigmoid activation function. The specific model architecture I used is visible in Table 1. The architectures for all three models were created using `tensorflow` and `keras` [19, 20].

Convolutional Autoencoder The Convolutional Autoencoder is similar to the Vanilla Autoencoder in its underlying structure, but it differs significantly in that it uses convolutional layers. Convolutional layers fundamentally differ from condensing layers as they use a small grid of weights, known as a kernel, to slide across the image and extract features. The layers condense the image by outputting a feature map from the sliding kernel by a set number of pixels (known as a stride) [4]. They are therefore better optimized for image applications, as they are designed to recognize features. The specific model architecture I used can be seen in Table 2.

Contractive Autoencoder The Contractive Autoencoder is different from the previous two in that it calculates the loss in a completely different fashion. Whereas both the Vanilla and Convolutional Autoencoders calculate loss by calculating the mean absolute error (discussed in section 3.2), the Contractive Autoencoder calculates loss by applying a penalty to the mean absolute error. This penalty accounts for the sensitivity of the latent representation to small changes in the input [5]. This way, the model is forced to learn the features of the images rather than getting confused by tiny changes due to effects like noise. The specific penalty I used is given by

$$L = MAE + 0.001 \cdot (D_{latent} \times (1 - D_{latent})^2 \times W^2) \tag{4}$$

where L is the total loss, MAE is the mean absolute error, D_{latent} is the latent dimension, and W are the weights of the last encoder step. The model architecture is given in Table 3.

Contractive Autoencoder	Layer	Input Shape	Neurons	Activation Function	Output Shape
Encoder	Layer 1	(80, 80, 1)			6400
	Layer 2	6400	512	ReLU	512
	Layer 3	512	256	ReLU	256
Latent Representation	Layer 4	256	64	ReLU	64
Decoder	Layer 1	64	256	ReLU	256
	Layer 2	256	512	ReLU	512
	Layer 3	512	6400	Sigmoid	6400
Reconstruction	Layer 4	6400			(80, 80, 1)

Table 3: Contractive Autoencoder Architecture

3.2 Loss Functions

Mean Absolute Error The first, and most trivial, way to calculate loss is to use the mean absolute error. For this method, the absolute value of the difference between each pixel in the image and its corresponding pixel in the reconstruction is calculated. The pixel errors are then averaged for each image. However, as seen in Figure 7, the noisy parts of the images are typically not reproduced by the model. Calculating the mean absolute error would not account for the error caused by noise, leading to reconstruction errors that cannot easily be used to detect anomalous galaxies.

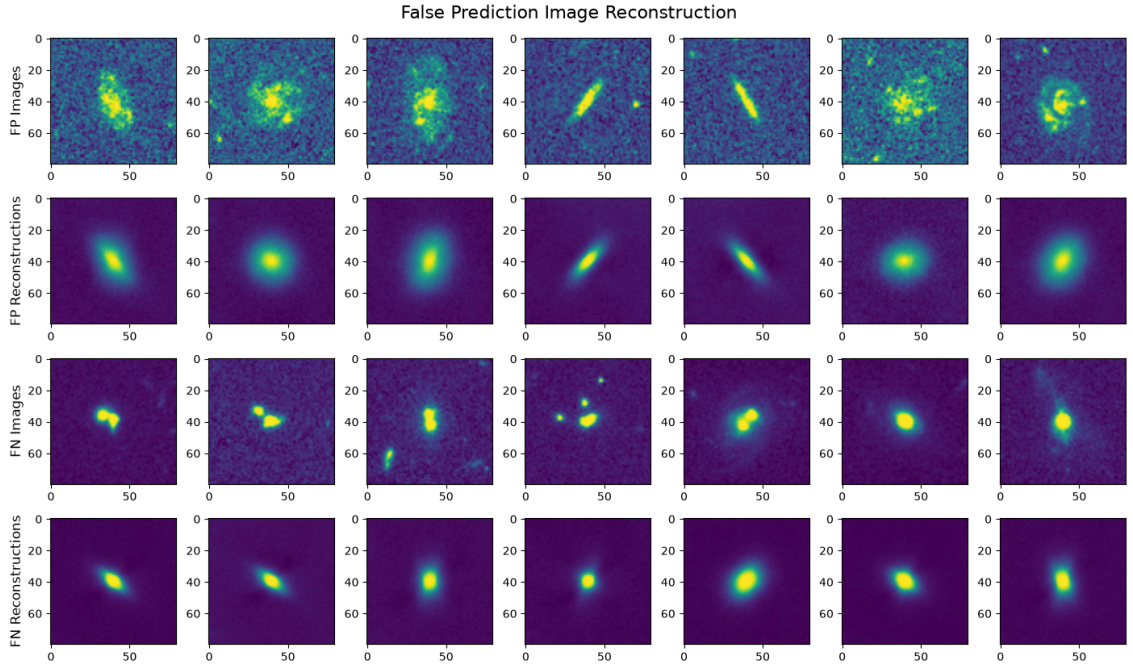

Figure 7: Image Reconstructions

χ^2-Loss To adjust for the problem of noisy images, I used Cochran's χ^2-Fit function, adapting it for my purposes [21]. The function is given by

$$\chi^2 = \sum \frac{(R - A)^2}{A} \tag{5}$$

where (for the purposes of this paper), R is the reconstruction value and A is the actual value. This should account for noisy pixels, as those pixels should have large A, leading to χ^2 decreasing. The typical χ^2-Fit formula divides by uncertainty, but I divided by the actual value A because a noisier pixel is likely to have a higher A value. I edited the formula slightly to take the mean of all values instead of the sum due to processing limitations.

Gaussian-weighted χ^2-Loss There is still another way, however, to decrease the effect that noisy pixels have on the reconstruction error. Since the pixels depicting a potential merger lie closer to the center of the image while noisy pixels lie on the edges, the reconstruction errors could be weighted more in the center of the image as opposed to the edges of it. To create these weights, I used a two-dimensional Gaussian curve normalized between 0.0 and 1.0, with the center of the image being the peak of the curve. After much experimentation, I chose $\sigma = 7.5$ to be the standard deviation of the curve, making it relatively steep. The χ^2 errors for each pixel would now be multiplied by their relative Gaussian weight, and then averaged to find the reconstruction error of the image.

3.3 Threshold Selection

For each configuration of denoising method, model, and loss function, the threshold was selected such that the F1-score would be maximized on the testing dataset. The F1-score is the harmonic mean of the precision and recall. Therefore, maximizing it would ensure that both the precision and recall would be balanced, without one increasing too much and decreasing the other [22, 23].

4 Results

4.1 Best Configuration Selection

To compare all 63 configurations, I selected three metrics: the area under the Precision-Recall Curve (AUC-PR), the area under the Receiver Operating Curve (AUC-ROC), and the F1-score. The Precision-Recall curve plots the precision, or the ratio of the number of true anomalous predictions to all anomalous predictions, on the y-axis and the recall, or the ratio of the number of true anomalous predictions to all anomalous images. As seen in Figure 8, the precision and recall are inversely related. Therefore, the threshold is selected to maximize the F1 score, as maximizing that score ensures that there is both a high recall and a high precision.

$$Precision = \frac{TP}{TP + FP} \tag{6}$$

$$Recall = TPR = \frac{TP}{TP + FN} \tag{7}$$

$$F_1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{8}$$

The Receiver Operating Curve plots the true positive rate (or the recall) on the y-axis and the false positive rate, the ratio of the falsely-labeled normal galaxies to all normal galaxies, on the x-axis. Random chance would provide a curve that has an area of 0.5 under it, whereas a model that performs perfectly has an area of 1.0 under it (See Figure 9).

$$FPR = \frac{FP}{FP + TN} \tag{9}$$

Out of all of these metrics, the F1-score is the most misleading. This is because, for anomaly detection, the score depends heavily on dataset-specific factors, namely the anomaly fraction and the difficulty of detection. On a dataset with a different anomaly fraction, a completely different F1-score could be produced [23]. Even though the anomaly fractions for all of the datasets are the same in this project and the same random state was used for each configuration, it is not useful to use a score that depends so heavily on the state of the training dataset. Therefore, it was only used to select the threshold for each configuration.

The AUC-ROC score is a better metric than the F1-score in that it does not change based on the dataset-specific factors. However, it hides a model's precision. If a model makes positive predictions far more often than it should, then it has a great AUC-ROC score because it selected all of the anomalies. However, its precision is poor because most of its positive predictions were false [24]. The goal of this project is to classify unknown galaxies, so a model that detects too

many false positives contaminates the classifications too much, making them useless for research purposes.

Therefore, the principal metric to be considered is the AUC-PR score. This score considers the true positives, false positives, and false negatives [24]. Its only drawback is that it doesn't consider the true negatives, but for our purposes this number is less important as our goal is to find merging galaxies.

The configuration with the highest AUC-PR score was the configuration that denoised images using the VisuShrink Wavelet method, trained and used a Contractive Autoencoder, and calculated loss using the Gaussian χ^2-Fit.

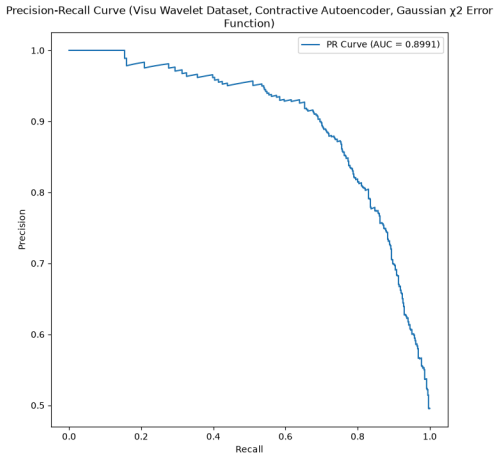

Figure 8: Precision-Recall Curve

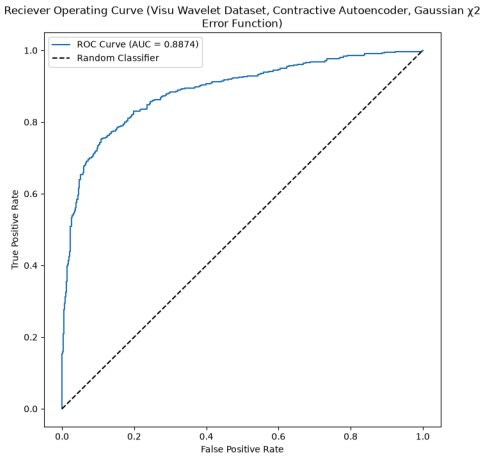

Figure 9: Receiver Operating Curve

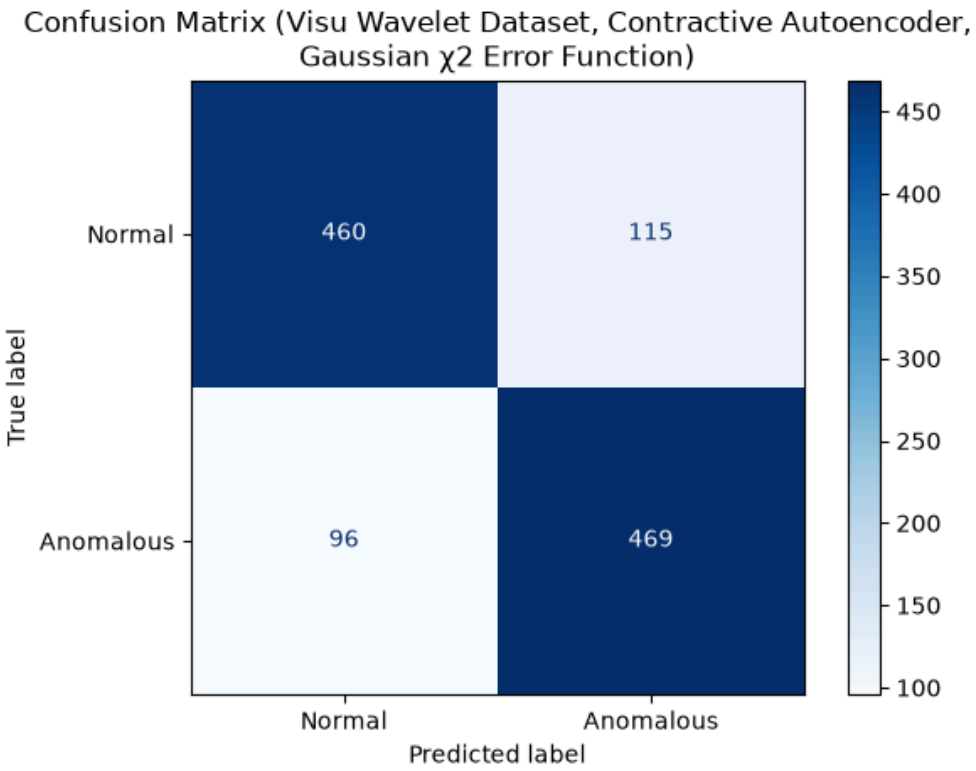

Figure 10: Confusion Matrix for the Best Configuration

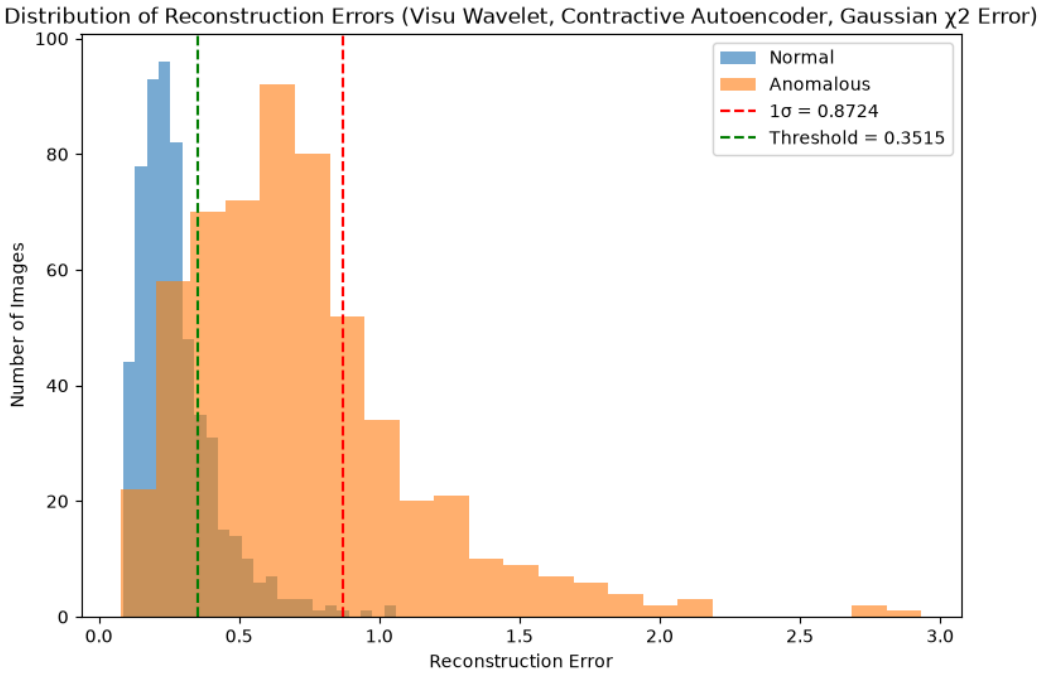

Figure 11: Reconstruction Error Distribution for the Best Configuration. As seen above, the peaks of the normal error distribution and the anomalous error distribution are separated, indicating the model can tell the two apart.

4.2 Configuration Comparison

The data from Appendix A was plotted on a graph. The x-axis depicts the AUC-ROC score while the y-axis depicts the AUC-PR score. Consistent with the observation made by Davis et al. (2006) that the two scores are proportional to each other, the AUC-PR score increased as the AUC-ROC score increased [22]. This is further emphasized by the positive linear trendline in the data. There are a few general observations to be made regarding the performance of certain configurations. As seen in Figure 12, the choice of denoising method seems to have had minimal impact on the configuration's overall performance. The choice of model and the loss function used seemed to have more of an impact on its performance. While the Contractive Autoencoder was the model used by the winning configuration, the most consistently well-performing model was the Vanilla Autoencoder. The Contractive and Convolutional Autoencoders were more dispersed throughout the entire dataset when it came to their scores (See Figure 13). By far, the best loss function was the Gaussian-weighted χ^2 Error, while the worst one was the Mean Absolute Error. The χ^2 Error was more evenly distributed, although it tended to perform better than the Mean Absolute Error (Figure 14).

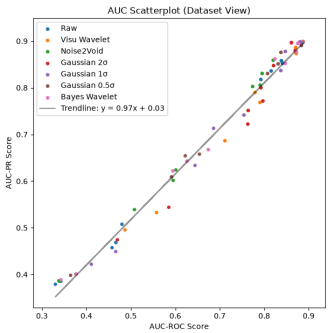

Figure 12: Configurations sorted by denoising method.

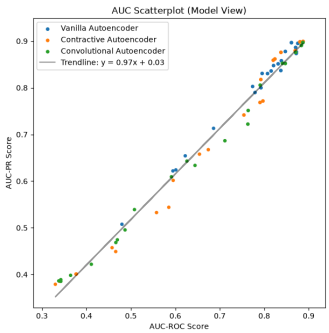

Figure 13: Configurations sorted by autoencoder type.

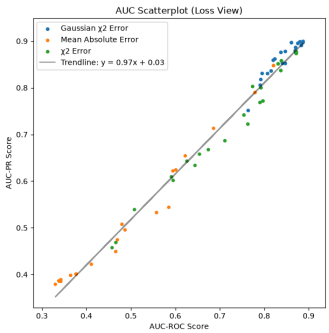

Figure 14: Configurations sorted by loss function.

4.3 Unknown Classifications

The winning configuration was finally used to classify 2763 of the unknown galaxies. Out of the 2763 galaxies, 531 were classified as "anomalous" while 2232 were classified as normal. As can be seen in Figure 15, visual inspections of some of the predictions show that the model made mostly correct predictions, and the incorrect predictions were mainly false negatives, reducing the likelihood of there being serious contamination in the anomalous galaxy set.

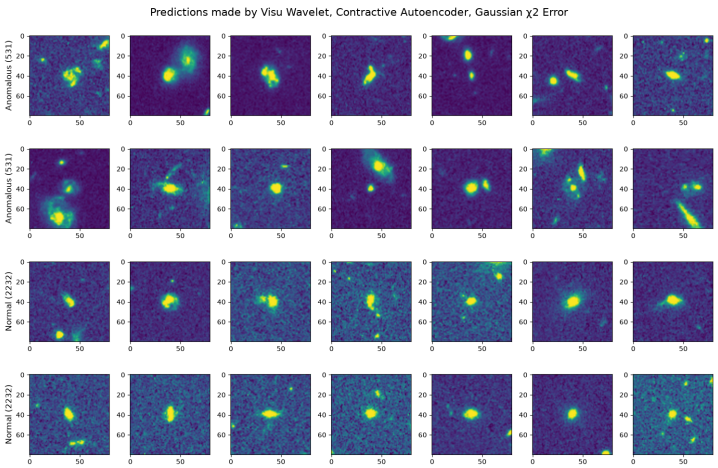

Figure 15: Unknown Galaxy Predictions (Not denoised to view morphological features)

5 Conclusion

In this paper, I set out to find the best autoencoder configuration to classify 204,082 unclassified galaxies as mergers or non-mergers. I analyzed 63 different autoencoder configurations, experimenting with denoising methods, model type, and loss function. I found that the best configuration denoised the data using the VisuShrink denoising method, Contractive Autoencoder, and Gaussian-weighted χ^2 Error. As a proof of concept, I used this configuration to classify 2763 unknown galaxies as mergers/non-mergers. All of the code, images, and figures are available on my GitHub Repository. Future projects could improve on this model by training it on more data and experimenting with other model architectures. Additionally, the model needs to be scaled up significantly in order to classify all 204,082 images.

Acknowledgments

I would like to thank the AstroPy, AstroQuery, SciPy, NumPy, TensorFlow, Keras, Scikit-Image, and Python teams for developing the software that allowed me to work on this project. I would also like to thank the European Space Agency for publicly releasing the Euclid large-scale survey data, as that data was crucial to this project. I would like to thank the Max Planck Institute for Astronomy and the Haus der Astronomie for providing me the resources necessary to start this project. Finally, I would most like to thank my mentor Dr. Niall Deacon for providing me guidance and helping me throughout this entire project.

References

[1] Euclid Collaboration et al. "Euclid Quick Data Release (Q1)". In: *Astron. Astrophys.* 711 (July 2026), A17.

[2] Euclid Collaboration et al. "Euclid Quick Data Release (Q1)". In: *Astron. Astrophys.* 711 (July 2026), A9.

[3] Federico Di Mattia et al. "A survey on GANs for Anomaly Detection". In: (June 2019). arXiv: 1906.11632 [cs.LG].

[4] Kate Storey-Fisher et al. "Anomaly detection in astronomical images with generative adversarial networks". In: (Dec. 2020). arXiv: 2012.08082 [astro-ph.GA].

[5] Dor Bank, Noam Koenigstein, and Raja Giryes. "Autoencoders". In: (Mar. 2020). arXiv: 2003.05991 [cs.LG].

[6] Raghavendra Chalapathy and Sanjay Chawla. "Deep learning for anomaly detection: A survey". In: (Jan. 2019). arXiv: 1901.03407 [cs.LG].

[7] A. Ginsburg et al. "astroquery: An Astronomical Web-querying Package in Python". In: *aj* 157, 98 (Mar. 2019), p. 98. DOI: 10.3847/1538-3881/aafc33. arXiv: 1901.04520 [astro-ph.IM].

[8] European Space Agency and Euclid Consortium. *Euclid Early Release Observations*. 2024.

[9] Astropy Collaboration et al. "Astropy: A community Python package for astronomy". In: *A&A* 558, A33 (Oct. 2013), A33. DOI: 10.1051/0004-6361/201322068. arXiv: 1307.6212 [astro-ph.IM].

[10] Astropy Collaboration et al. "The Astropy Project: Building an Open-science Project and Status of the v2.0 Core Package". In: *AJ* 156.3, 123 (Sept. 2018), p. 123. DOI: 10.3847/1538-3881/aabc4f. arXiv: 1801.02634 [astro-ph.IM].

[11] Astropy Collaboration et al. "The Astropy Project: Sustaining and Growing a Community-oriented Open-source Project and the Latest Major Release (v5.0) of the Core Package". In: *ApJ* 935.2, 167 (Aug. 2022), p. 167. DOI: 10.3847/1538-4357/ac7c74. arXiv: 2206.14220 [astro-ph.IM].

[12] Euclid Collaboration et al. *Euclid Quick Data Release (Q1): VIS processing and data products*. 2025. arXiv: 2503.15303 [astro-ph.IM]. URL: https://arxiv.org/abs/2503.15303.

[13] G Nurbaeva et al. "On the effect of image denoising on galaxy shape measurements". In: (June 2011). arXiv: 1106.0927 [astro-ph.CO].

[14] V Roscani et al. “A comparative analysis of denoising algorithms for extragalactic imaging surveys”. In: *Astron. Astrophys.* 643 (Nov. 2020), A43.

[15] Pauli Virtanen et al. “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python”. In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[16] S.G. Chang, B. Yu, and Martin Vetterli. “Adaptive wavelet thresholding for image denoising and compression”. In: *IEEE Transactions on Image Processing* 9 (Feb. 2000), pp. 1532–1546. DOI: 10.1109/83.862633.

[17] F. Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[18] Alexander Krull, Tim-Oliver Buchholz, and Florian Jug. “Noise2Void - learning denoising from single noisy images”. In: (Nov. 2018). arXiv: 1811.10980 [cs.CV].

[19] Martín Abadi et al. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. 2015. URL: https://www.tensorflow.org/.

[20] François Chollet et al. *Keras*. https://keras.io. 2015.

[21] William G Cochran. “The χ^2 test of goodness of fit”. In: *Ann. Math. Stat.* 23.3 (Sept. 1952), pp. 315–345.

[22] Jesse Davis and Mark Goadrich. “The relationship between Precision-Recall and ROC curves”. In: *Proceedings of the 23rd International Conference on Machine Learning*. ICML ’06. Pittsburgh, Pennsylvania, USA: Association for Computing Machinery, 2006, pp. 233–240. ISBN: 1595933832. DOI: 10.1145/1143844.1143874. URL: https://doi.org/10.1145/1143844.1143874.

[23] Damien Fourure et al. “Anomaly detection: How to artificially increase your F1-score with a biased evaluation protocol”. In: (June 2021). arXiv: 2106.16020 [cs.LG].

[24] Jan Brabec and Lukas Machlica. “Bad practices in evaluation methodology relevant to class-imbalanced problems”. In: (Dec. 2018). arXiv: 1812.01388 [cs.LG].

Appendix A Performance scores of all configurations

The following table shows the AUC-PR score, AUC-ROC score, and the F1-Score for each of the 63 configurations. It is sorted in descending order by the AUC-PR score.

Dataset	Model	Loss Function	AUC-PR	AUC-ROC	F1-Score
Visu Wavelet	Contractive Autoencoder	Gaussian χ^2 Error	0.8991	0.8874	0.8164
Gaussian 1σ	Contractive Autoencoder	Gaussian χ^2 Error	0.8983	0.8804	0.8245
Gaussian 1σ	Convolutional Autoencoder	Gaussian χ^2 Error	0.8968	0.8870	0.8242
Gaussian 2σ	Vanilla Autoencoder	Gaussian χ^2 Error	0.8966	0.8612	0.8122
Bayes Wavelet	Vanilla Autoencoder	Gaussian χ^2 Error	0.8951	0.8757	0.8344
Gaussian 0.5σ	Convolutional Autoencoder	Gaussian χ^2 Error	0.8906	0.8835	0.8172
Visu Wavelet	Vanilla Autoencoder	Gaussian χ^2 Error	0.8862	0.8707	0.8003
Gaussian 2σ	Contractive Autoencoder	Gaussian χ^2 Error	0.8793	0.8698	0.8165
Gaussian 1σ	Vanilla Autoencoder	Gaussian χ^2 Error	0.8777	0.8475	0.7816
Bayes Wavelet	Vanilla Autoencoder	χ^2 Error	0.8768	0.8722	0.8142
Visu Wavelet	Convolutional Autoencoder	Gaussian χ^2 Error	0.8759	0.8706	0.8010
Gaussian 0.5σ	Contractive Autoencoder	Gaussian χ^2 Error	0.8758	0.8376	0.7938
Visu Wavelet	Vanilla Autoencoder	χ^2 Error	0.8735	0.8720	0.8091
Bayes Wavelet	Contractive Autoencoder	Gaussian χ^2 Error	0.8616	0.8240	0.7761
Noise2Void	Contractive Autoencoder	Gaussian χ^2 Error	0.8589	0.8203	0.7689
Raw	Vanilla Autoencoder	χ^2 Error	0.8578	0.8386	0.7826
Raw	Convolutional Autoencoder	Gaussian χ^2 Error	0.8523	0.8411	0.7881
Bayes Wavelet	Convolutional Autoencoder	Gaussian χ^2 Error	0.8520	0.8474	0.7890
Gaussian 0.5σ	Vanilla Autoencoder	χ^2 Error	0.8512	0.8313	0.7727
Gaussian 2σ	Vanilla Autoencoder	Mean Absolute Error	0.8475	0.8210	0.7584
Gaussian 1σ	Vanilla Autoencoder	χ^2 Error	0.8369	0.8374	0.7795
Raw	Vanilla Autoencoder	Gaussian χ^2 Error	0.8359	0.8156	0.7663
Noise2Void	Vanilla Autoencoder	Gaussian χ^2 Error	0.8306	0.7953	0.7544

Gaussian 0.5σ	Vanilla Autoencoder	Gaussian χ^2 Error	0.8304	0.8075	0.7528
Raw	Contractive Autoencoder	Gaussian χ^2 Error	0.8176	0.7924	0.7455
Noise2Void	Convolutional Autoencoder	Gaussian χ^2 Error	0.8056	0.7909	0.7364
Noise2Void	Vanilla Autoencoder	χ^2 Error	0.8027	0.7739	0.7224
Gaussian 2σ	Vanilla Autoencoder	χ^2 Error	0.7999	0.7927	0.7353
Visu Wavelet	Vanilla Autoencoder	Mean Absolute Error	0.7898	0.7793	0.7295
Gaussian 2σ	Contractive Autoencoder	χ^2 Error	0.7716	0.7971	0.7655
Visu Wavelet	Contractive Autoencoder	χ^2 Error	0.7685	0.7905	0.7562
Gaussian 2σ	Convolutional Autoencoder	Gaussian χ^2 Error	0.7513	0.7639	0.7342
Gaussian 1σ	Contractive Autoencoder	χ^2 Error	0.7416	0.7543	0.7384
Gaussian 2σ	Convolutional Autoencoder	χ^2 Error	0.7221	0.7632	0.7327
Gaussian 1σ	Vanilla Autoencoder	Mean Absolute Error	0.7131	0.6860	0.6620
Visu Wavelet	Convolutional Autoencoder	χ^2 Error	0.6865	0.7115	0.7055
Bayes Wavelet	Contractive Autoencoder	χ^2 Error	0.6674	0.6742	0.6889
Gaussian 0.5σ	Contractive Autoencoder	χ^2 Error	0.6578	0.6543	0.6758
Gaussian 0.5σ	Vanilla Autoencoder	Mean Absolute Error	0.6542	0.6223	0.6620
Bayes Wavelet	Convolutional Autoencoder	χ^2 Error	0.6429	0.6261	0.6639
Gaussian 1σ	Convolutional Autoencoder	χ^2 Error	0.6337	0.6443	0.6713
Noise2Void	Vanilla Autoencoder	Mean Absolute Error	0.6241	0.6012	0.6620
Bayes Wavelet	Vanilla Autoencoder	Mean Absolute Error	0.6219	0.5946	0.6620
Gaussian 0.5σ	Convolutional Autoencoder	χ^2 Error	0.6093	0.5912	0.6620
Noise2Void	Contractive Autoencoder	χ^2 Error	0.6016	0.5952	0.6620
Gaussian 2σ	Contractive Autoencoder	Mean Absolute Error	0.5440	0.5853	0.6715
Noise2Void	Convolutional Autoencoder	χ^2 Error	0.5391	0.5081	0.6620
Visu Wavelet	Contractive Autoencoder	Mean Absolute Error	0.5327	0.5577	0.6637
Raw	Vanilla Autoencoder	Mean Absolute Error	0.5077	0.4799	0.6620
Visu Wavelet	Convolutional Autoencoder	Mean Absolute Error	0.4955	0.4869	0.6631
Gaussian 2σ	Convolutional Autoencoder	Mean Absolute Error	0.4744	0.4696	0.6620
Raw	Convolutional Autoencoder	χ^2 Error	0.4685	0.4662	0.6620
Raw	Contractive Autoencoder	χ^2 Error	0.4575	0.4577	0.6624
Gaussian 1σ	Contractive Autoencoder	Mean Absolute Error	0.4492	0.4659	0.6620
Gaussian 1σ	Convolutional Autoencoder	Mean Absolute Error	0.4220	0.4111	0.6620
Bayes Wavelet	Contractive Autoencoder	Mean Absolute Error	0.4011	0.3775	0.6620
Gaussian 0.5σ	Contractive Autoencoder	Mean Absolute Error	0.4010	0.3764	0.6620
Gaussian 0.5σ	Convolutional Autoencoder	Mean Absolute Error	0.3983	0.3644	0.6620
Bayes Wavelet	Convolutional Autoencoder	Mean Absolute Error	0.3890	0.3424	0.6620
Noise2Void	Convolutional Autoencoder	Mean Absolute Error	0.3864	0.3378	0.6620
Noise2Void	Contractive Autoencoder	Mean Absolute Error	0.3855	0.3395	0.6620
Raw	Convolutional Autoencoder	Mean Absolute Error	0.3854	0.3424	0.6620
Raw	Contractive Autoencoder	Mean Absolute Error	0.3791	0.3301	0.6620